Comparing Crank-Nicolson, PINNs and PIKANs for Black-Scholes Option Pricing

Dylan Sinnott
dylansinnott572@gmail.com

1 Introduction

Options are financial derivatives that give a buyer the right to buy (call option) or sell (put option) an asset at a fixed price at some time in the future. European and American options differ in that European options grant the right to buy or sell the asset at the exercise price only at the maturity date, whereas American options can be exercised any time up to and including the date of maturity.

Determining the value of these options contracts depends on several factors including strike price, time-to-maturity, volatility and the risk-free interest rate. Partial Differential Equations (PDEs) can be used to determine the price of options; by solving the PDE, you end up with a function into which you can plug the factors listed above and it will then deliver a fair option price.

One such PDE is the *Black-Scholes* equation,

$$\frac{\partial V}{\partial t} + \frac{1}{2}S^2\sigma^2\frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0. \tag{1}$$

where V is the option price, t is time, r is the risk-free interest rate, σ is volatility and S is the underlying stock price. This equation comes with a set of terminal and boundary conditions for European call options,

$$V(S,T) = \max(S-K, 0)$$

and

$$V(0,t) = 0,\ V(S\to\infty, t) \approx S_{max} - Ke^{-r(T-t)}$$

where S_{max} represents the upper stock-price boundary used in the numerical approximation.

For European call options this equation can be solved analytically, giving a price function for European call options. However the Black-Scholes model is not the only option pricing PDE that exists, and it does have its limitations, including the assumption of constant volatility. For this reason other models exist, such as the Heston model, Monte Carlo simulations, binomial trees and many more.

Some of these models do not have feasible analytical solutions, including the Black-Scholes model for American options, and for that reason finite-difference methods are often employed to

solve these equations. However in recent times there have been significant advances in machine-learning frameworks, including the development of Physics-Informed Neural Networks (PINNs) and Physics-Informed Kolmogorov-Arnold Networks (PIKANs). This project investigates whether PINNs or PIKANs can provide competitive accuracy or efficiency when compared with a classical finite-difference method for European option pricing under the Black-Scholes equation.

2 Mathematical Background

In this project I will test the Crank-Nicolson method, PINNs and PIKANs at solving (1). The parameters for these models include the aforementioned risk-free interest rate r and volatility σ, as well as the initial underlying stock price, maturity time T and strike price K, which denotes the price for which the contract can be exercised.

Before introducing these models it is convenient to work with (1) in terms of time to maturity. Introduce a new variable,

$$\tau = T - t.$$

In this new variable τ the Black-Scholes equation reads,

$$\frac{\partial V}{\partial \tau} = \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV. \tag{2}$$

while the terminal condition has become an initial condition,

$$V(S, 0) = \max(S - K, 0).$$

and the boundary conditions remain much the same, only with our new time to maturity variable τ,

$$V(0, \tau) = 0,\ V(S \to \infty, \tau) \approx S_{\max} - Ke^{-r\tau}.$$

The analytical Black-Scholes formula resulting from solving this PDE will be used as a benchmark against the numerical and machine learning approaches. Solving this equation in this form gives,

$$V(S, \tau) = S\Phi(d_1) - Ke^{-r\tau}\Phi(d_2), \tag{3}$$

where

$$d_1 = \frac{\ln(S/K) + \left(r + \frac{1}{2}\sigma^2\right)\tau}{\sigma\sqrt{\tau}},$$

and

$$d_2 = d_1 - \sigma\sqrt{\tau}.$$

Here, Φ denotes the cumulative distribution function of the standard normal distribution.

3 Methods

3.1 Crank-Nicolson Method

The Crank-Nicolson method is a finite difference method (FDM) designed to work for parabolic PDEs such as the Black-Scholes equation. In comparison to fully implicit or explicit FDMs it takes an average of the two, solving the PDE over a fixed grid of values.

The spatial discretisation used in solving the Black-Scholes involved partitioning the spatial domain $[0, S_{max}]$ into M uniform intervals. The spatial step size is then,

$$\Delta S = \frac{S_{max}}{M}.$$

The numerical solution at stock price S_i and time-to-maturity τ_j is denoted

$$V_i^j = V(S_i, \tau_j)$$

The first and second partial derivatives are then computed using second-order central finite difference schemes,

$$\left.\frac{\partial V}{\partial S}\right|_{S_i} \approx \frac{V_{i+1}^j - V_{i-1}^j}{2\Delta S}$$

and

$$\left.\frac{\partial^2 V}{\partial S^2}\right|_{S_i} \approx \frac{V_{i+1}^j - 2V_i^j + V_{i-1}^j}{(\Delta S)^2}$$

The temporal domain $[0, T]$ was split into N uniform time steps with

$$\Delta\tau = \frac{T}{N}.$$

To state the Crank-Nicolson temporal discretisation first define the differential operator of the Black-Scholes equation,

$$L(V) := \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV. \tag{4}$$

The Crank-Nicolson method averages this operator between two consecutive time intervals,

$$\frac{V_i^{j+1} - V_i^j}{\Delta\tau} = \frac{1}{2}\left[L(V_i^{j+1}) + L(V_i^j)\right]. \tag{5}$$

This leads to a tridiagonal linear system at each time step,

$$AV^{j+1} = BV^j + b^j \tag{6}$$

where A and B are tridiagonal matrices determined by the grid sizes and model parameters and b^j contains the boundary condition contributions.

3.2 Physics-Informed Neural Network

The Physics-Informed Neural Network (PINN) uses a standard multilayer perceptron to approximate the Black-Scholes option price. The network is trained not only to fit boundary and initial conditions, but also to satisfy the Black-Scholes PDE inside the computational domain. This allows the model to learn an approximation of the option price surface without requiring labelled option prices at every point.

The PINN approximates the option price as

$$V_\theta(S, \tau) \approx V(S, \tau),$$

where S is the asset price and τ is the time to maturity. Before passing the inputs into the neural network, both variables are scaled to the interval $[0, 1]$:

$$\tilde{S} = \frac{S}{S_{\max}}, \qquad \tilde{\tau} = \frac{\tau}{T}.$$

The network input is therefore

$$x = (\tilde{S}, \tilde{\tau}).$$

The PINN was implemented as a fully connected feedforward neural network with three hidden layers. Each hidden layer used the hyperbolic tangent activation function. The output of the network was multiplied by a price scaling factor, so that the neural network output was on a similar scale to the option prices being approximated.

Component	Value
Hidden layers	3
Activation function	`Tanh`
Input scaling	$S/S_{\max}$, τ/T
Price scaling	K
Epochs	5000
Optimiser	Adam
Learning rate	10^{-3}

Table 1: Baseline PINN architecture and training settings.

The PINN is trained by penalising the Black-Scholes PDE residual. For the neural approximation $V_\theta(S, \tau)$, the residual is

$$\mathcal{R}_\theta(S, \tau) = \frac{\partial V_\theta}{\partial \tau} - \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V_\theta}{\partial S^2} - rS\frac{\partial V_\theta}{\partial S} + rV_\theta.$$

The residual is zero when the approximation satisfies the Black-Scholes PDE exactly. The derivatives appearing in this expression are computed using automatic differentiation.

The total PINN loss is made up of the PDE loss, the initial condition loss, and the lower and upper boundary condition losses:

$$\mathcal{L} = \mathcal{L}_{PDE} + \mathcal{L}_{IC} + \mathcal{L}_{LB} + \mathcal{L}_{UB}.$$

Here, $\mathcal{L}_{PDE}$ penalises violations of the PDE in the interior of the domain, $\mathcal{L}_{IC}$ enforces the payoff condition at expiry, and $\mathcal{L}_{LB}$ and $\mathcal{L}_{UB}$ enforce the lower and upper boundary conditions. The same loss structure is later used for the PIKAN, allowing the two neural approaches to be compared under the same Black-Scholes setup.

3.3 Physics-Informed Kolmogorov-Arnold Networks

The Physics-Informed Kolmogorov-Arnold Network (PIKAN) uses the same physics-informed training framework as the PINN, but replaces the standard multilayer perceptron with a Kolmogorov-Arnold Network (KAN). In a standard PINN, the trainable parameters are weights and biases in linear layers. In a KAN, the trainable components are the spline-based functions used in place of activation functions.

The PIKAN approximates the option price as

$$V_\theta(S, \tau) \approx V(S, \tau),$$

where S is the asset price and τ is the time to maturity. Before passing the inputs into the KAN, both variables are scaled to the interval $[0, 1]$:

$$\tilde{S} = \frac{S}{S_{\max}}, \qquad \tilde{\tau} = \frac{\tau}{T}.$$

The network input is therefore

$$x = (\tilde{S}, \tilde{\tau}).$$

The KAN was implemented using PyKAN. The network width was specified by the list of layer sizes, while the spline grid size was controlled by the parameter `grid`. Cubic splines were used by setting the spline degree to $k = 3$. Symbolic activation functions were disabled so that the model acted purely as a numerical function approximator.

As with the PINN, the PIKAN is trained by penalising the Black-Scholes PDE residual. For the neural approximation $V_\theta(S, \tau)$, the residual is

$$\mathcal{R}_\theta(S, \tau) = \frac{\partial V_\theta}{\partial \tau} - \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V_\theta}{\partial S^2} - rS\frac{\partial V_\theta}{\partial S} + rV_\theta.$$

The residual is zero when the approximation satisfies the Black-Scholes PDE exactly. The derivatives appearing in this expression are computed using automatic differentiation.

The total PIKAN loss uses the same structure as the PINN loss:

$$\mathcal{L} = \mathcal{L}_{PDE} + \mathcal{L}_{IC} + \mathcal{L}_{LB} + \mathcal{L}_{UB}.$$

Here, $\mathcal{L}_{PDE}$ penalises violations of the PDE in the interior of the domain, $\mathcal{L}_{IC}$ enforces the payoff condition at expiry, and $\mathcal{L}_{LB}$ and $\mathcal{L}_{UB}$ enforce the lower and upper boundary conditions. Since the PIKAN and PINN use the same physics-informed loss terms and evaluation procedure, the comparison mainly reflects the effect of replacing the MLP architecture with a KAN architecture.

4 Experimental Setup

All methods were evaluated against the analytical Black-Scholes solution (3). The evaluation grid was kept the same for all methods, with stock-price and time-to-maturity values evaluated over the same fixed grid.

Seeds were fixed across all of the method notebooks. Both NumPy and PyTorch seed values were set to 42 so that experiments were easily reproducible.

The neural networks were trained with the following set of parameters,

Parameter	Value
Strike Price K	100
Maturity T	1
Risk-free rate r	0.05
Volatility σ	0.2
Maximum stock price S_{max}	300

Table 2: Experimental Setup

Five variants of PINNs were tested, a baseline PINN with a [2, 32, 32, 32, 1] architecture, a wider PINN with a [2, 64, 64, 64, 1] architecture, a wider PINN with a learning rate scheduler, a wider PINN with a learning rate scheduler and a modified loss function of the form,

$$\mathcal{L} = \mathcal{L}_{PDE} + 5 \cdot \mathcal{L}_{IC} + \mathcal{L}_{LB} + \mathcal{L}_{UB},$$

and the baseline PINN with this modified loss function. All were run for 5000 epochs.

One PIKAN variant was used. It was run on two different training lengths, once for 3000 epochs and once for 1000 epochs.

5 Results

This section presents the numerical results for the Crank-Nicolson method, the baseline PINN, and the PIKAN. Each method is evaluated against the analytical Black-Scholes solution on the same evaluation grid. Performance is compared using mean squared error, root mean squared error, mean absolute error, maximum absolute error, relative L^2 error and runtime.

5.1 Crank-Nicolson Results

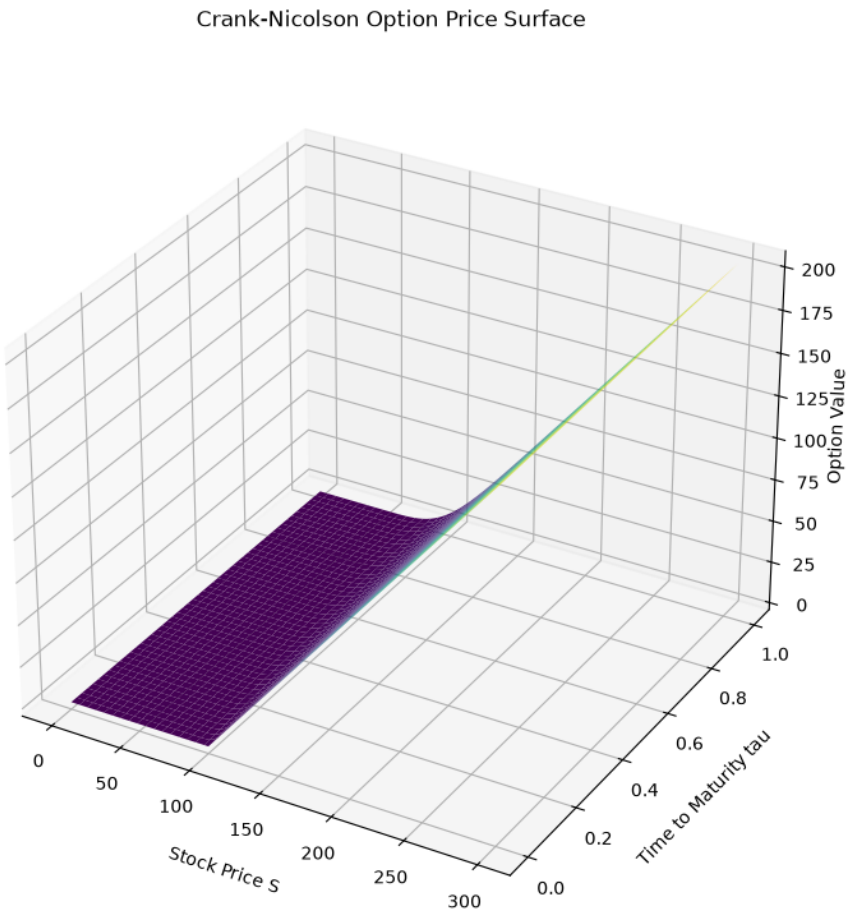


Figure 1: Crank-Nicolson option price surface for the European call option.

Figure 1 shows the option price surface generated by the Crank-Nicolson method. It very closely resembles the surface generated by the analytical Black-Scholes equation, indicating the Crank-Nicolson method does a good job of approximating the solution.

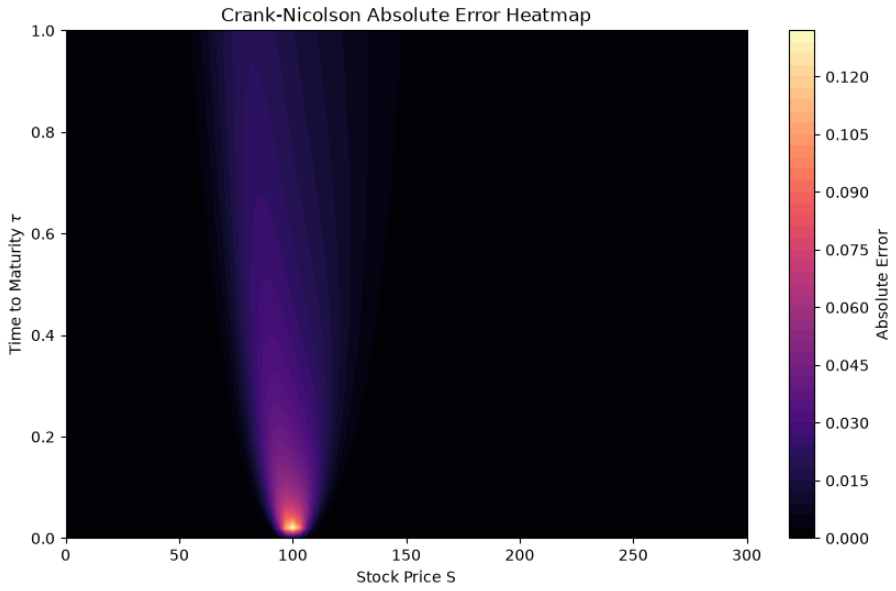


Figure 2: Crank-Nicolson Absolute Error Surface

The error surface indicates that the Crank-Nicolson method extremely accurately approximates the Black-Scholes solution, with the only non-negligible errors seemingly resulting near the strike price $K = 100$.

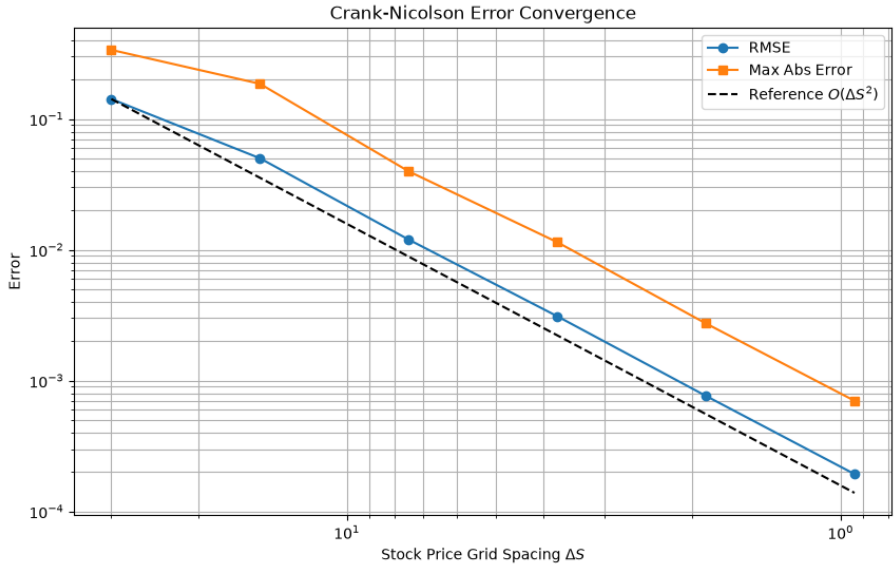


Figure 3: Crank-Nicolson Convergence Map

The Crank-Nicolson solver converges at a very similar rate to the theoretical $O(\Delta S^2)$ convergence rate.

Method	Architecture / Grid	Params	Epochs	Training Points	Eval. Grid	Runtime	Rt./Epoch	MSE	RMSE	MAE	Max Error	Rel. L^2
Crank-Nicolson	$M = 100$, $N = 100$	0	N/A	N/A	100×50	0.34	N/A	0.000096	0.009815	0.003887	0.129387	0.000102

Table 3: Crank-Nicolson Method Results.

The Crank-Nicolson method produced a smooth price surface closely matching the analytical

Black-Scholes solution. The absolute error remained small across most of the domain, with the largest errors occurring near the strike region. This behaviour is expected for finite difference methods applied to option pricing problems.

5.2 PINN Results

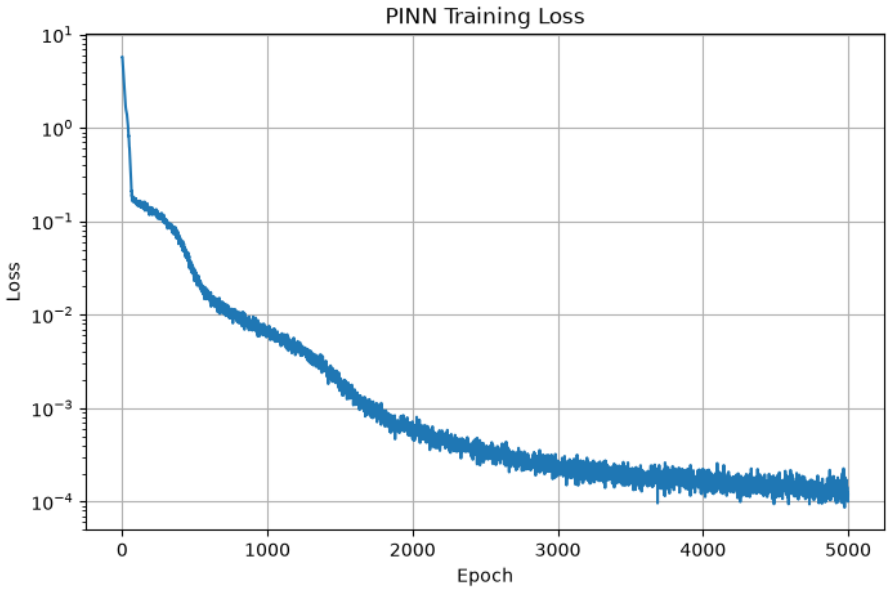


Figure 4: PINN Training Loss Curve

The (baseline) PINN loss curve shows the loss rapidly decreasing initially, gradually smoothing out as the number of epochs increases, as expected.

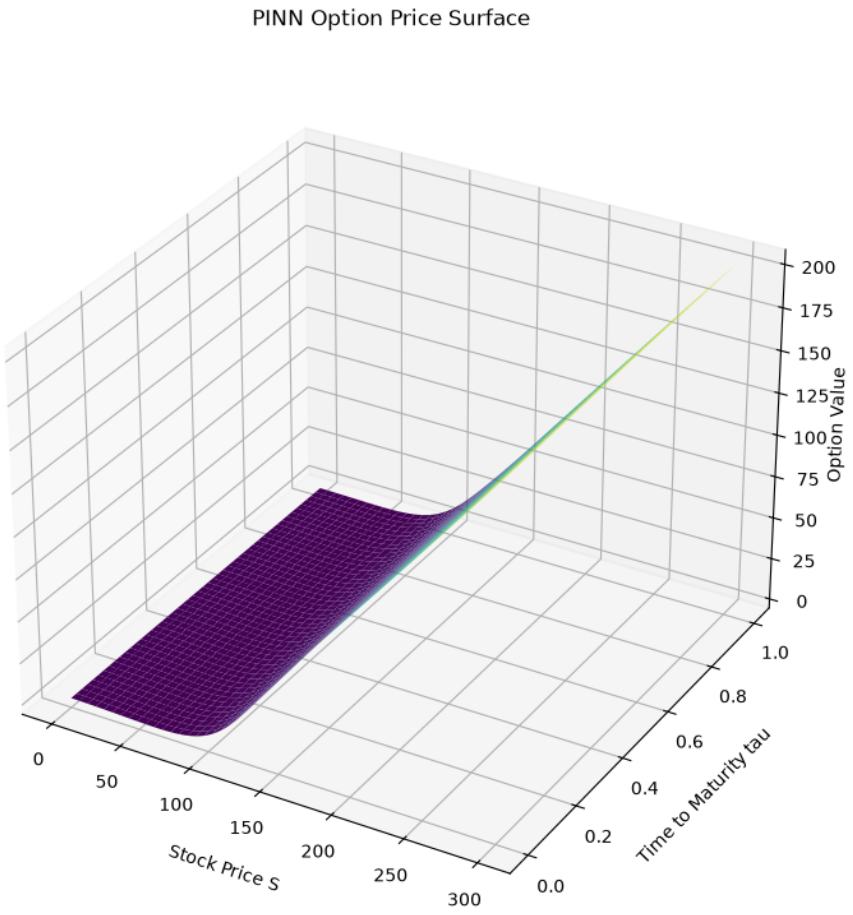


Figure 5: PINN Price Surface

The (baseline) PINN price surface closely resembles the theoretical Black-Scholes surface, indicating that the PINN has broadly captured the nature of the PDE.

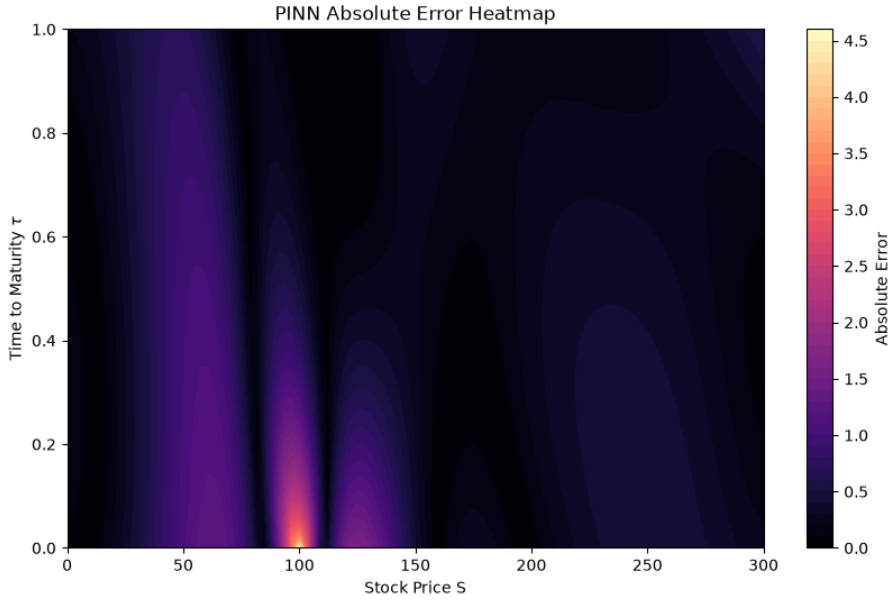


Figure 6: PINN Absolute Error Surface

Unlike the Crank-Nicolson absolute error heatmap, the (baseline) PINN exhibits some absolute error even in the far field conditions, not just around the strike price. The magnitude of the absolute errors is also considerably higher than those seen in the Crank-Nicolson method.

Model	Architecture	Scheduler	Loss	Epochs	Runtime	Rt./Epoch	MSE	RMSE	MAE	Max Error	Rel. L^2
Baseline PINN	$[2, 32, 32, 32, 1]$	No	Standard	5000	33.70	0.006470	0.250624	0.500623	0.363268	4.505993	0.005180
Wider PINN	$[2, 64, 64, 64, 1]$	No	Standard	5000	37.00	0.007400	0.482299	0.694477	0.562019	5.379873	0.007186
Wider PINN + LR Scheduler	$[2, 64, 64, 64, 1]$	StepLR	Standard	5000	38.50	0.007700	1.963900	1.401392	0.992939	7.813058	0.014500
Wider PINN + LR Scheduler + Modified Loss	$[2, 64, 64, 64, 1]$	StepLR	Weighted IC	5000	37.50	0.007500	0.298793	0.546620	0.429870	3.887356	0.005656
Baseline PINN + Modified Loss	$[2, 32, 32, 32, 1]$	No	Weighted IC	5000	26.80	0.005360	0.427081	0.653515	0.590918	3.244408	0.006762

Table 4: Comparison of PINN model variants.

The baseline PINN performed best across all error metrics except maximum error, where it was only outperformed by the models with weighted initial condition modified loss functions. All PINN variants recorded longer runtimes and greater errors across all metrics than the Crank-Nicolson method.

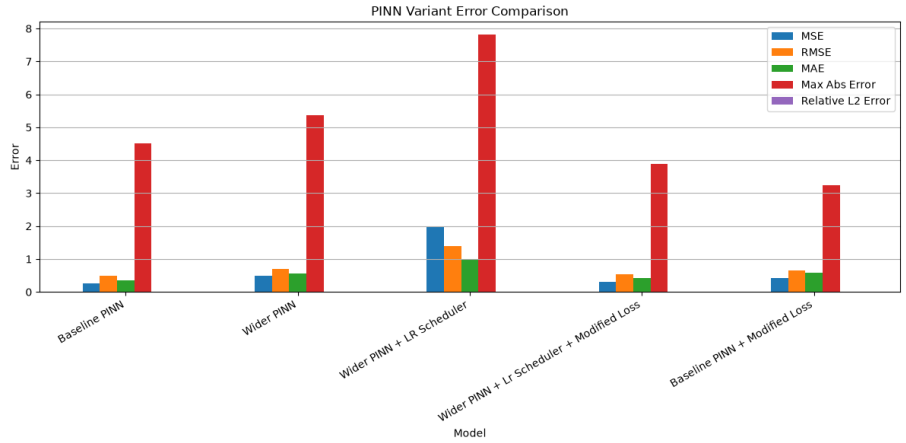


Figure 7: PINN Error Comparison

As is evident in Table 4, the baseline PINN performs best across all error metrics except max absolute error. For this reason, this model is chosen for the comparison between Crank-Nicolson, PINNs and PIKANs.

The PINN was able to learn the general structure of the Black-Scholes price surface. However, compared with the Crank-Nicolson method, the error was larger and the runtime was substantially higher due to the cost of neural network training. The largest errors again appeared near the strike region, reflecting the difficulty of approximating the payoff kink at expiry.

5.3 PIKAN Results

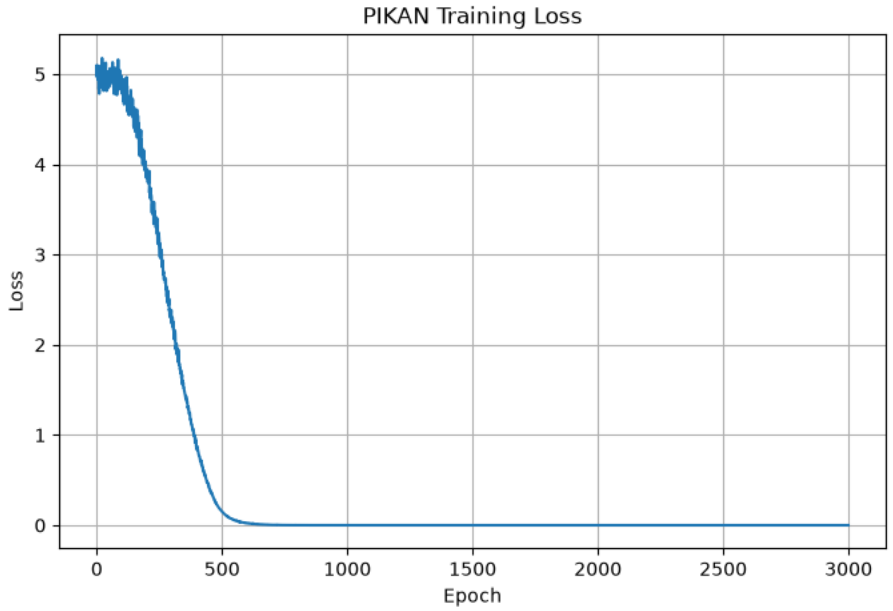


Figure 8: PIKAN Training Loss Curve

The loss curve appears to flatten around 800 epochs. This motivated me to tune the PIKAN model by training it for only 1000 epochs instead of 3000.

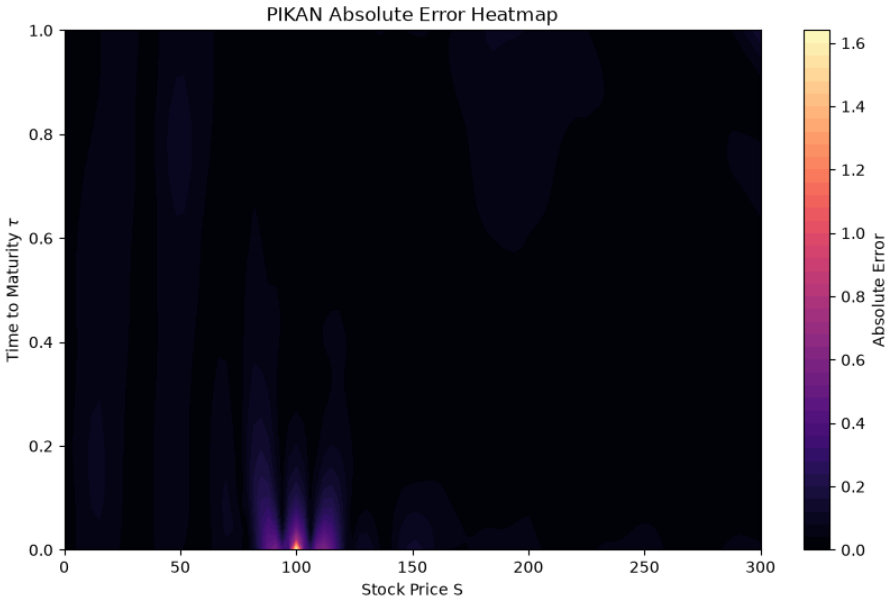


Figure 9: PIKAN Absolute Error Surface

The PIKAN absolute error surface more closely resembles that of the Crank-Nicolson surface than the PINN surface. Some localised errors occur near the strike price but errors are low in the far field conditions. Again however, the errors are far greater than those of the Crank-Nicolson solver.

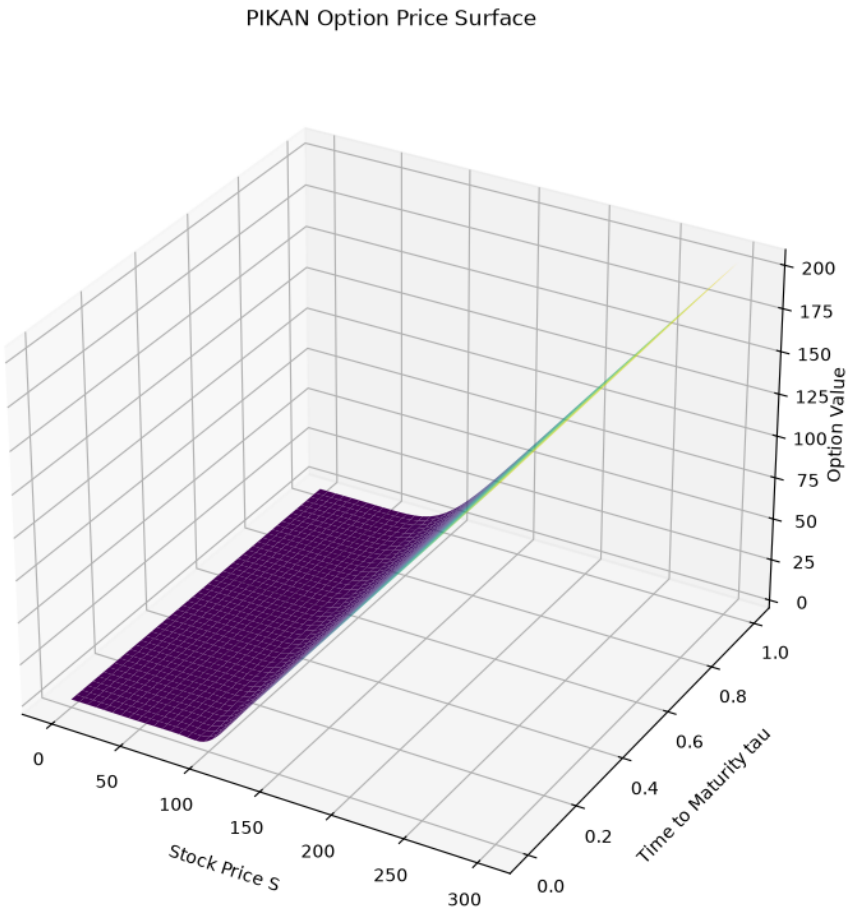


Figure 10: PIKAN Price Surface

Again, the PIKAN accurately captures the shape of the Black-Scholes price surface.

Method	Architecture / Grid	Params	Epochs	Training Points	Runtime	Rt./Epoch	MSE	RMSE	MAE	Max Error	Rel. L^2
PIKAN (1000 epochs)	$[2, 16, 16, 1]$, grid $= 5$, degree $= 3$	4256	1000	$N_f = 300$, $N_{ic} = 150$, $N_{bc} = 150$	88.10	0.088100	0.082062	0.286464	0.204735	3.646942	0.002964
PIKAN	$[2, 16, 16, 1]$, grid $= 5$, degree $= 3$	4256	3000	$N_f = 300$, $N_{ic} = 150$, $N_{bc} = 150$	257.60	0.085870	0.003373	0.058075	0.033826	1.629618	0.000601

Table 5: Comparison of PIKAN models trained for different numbers of epochs.

The PIKAN also recovered the overall shape of the analytical price surface. It recorded lower error metrics than the PINN but higher error metrics than the Crank-Nicolson method.

5.4 Method Comparison

Model	Architecture	Epochs	Runtime	Rt./Epoch	MSE	RMSE	MAE	Max Error	Rel. L^2	Scheduler	Loss
Baseline PINN	$[2, 32, 32, 32, 1]$	5000	33.70	0.006470	0.250624	0.500623	0.363268	4.505993	0.005180	No	Standard physics-informed loss
PIKAN	$[2, 16, 16, 1]$, grid $= 5$, degree $= 3$	3000	257.60	0.085870	0.003373	0.058075	0.033826	1.629618	0.000601	No	Standard physics-informed loss
PIKAN (1000 epochs)	$[2, 16, 16, 1]$, grid $= 5$, degree $= 3$	1000	88.10	0.088100	0.082062	0.286464	0.204735	3.646942	0.002964	No	Standard physics-informed loss

Table 6: Comparison between the baseline PINN and PIKAN models.

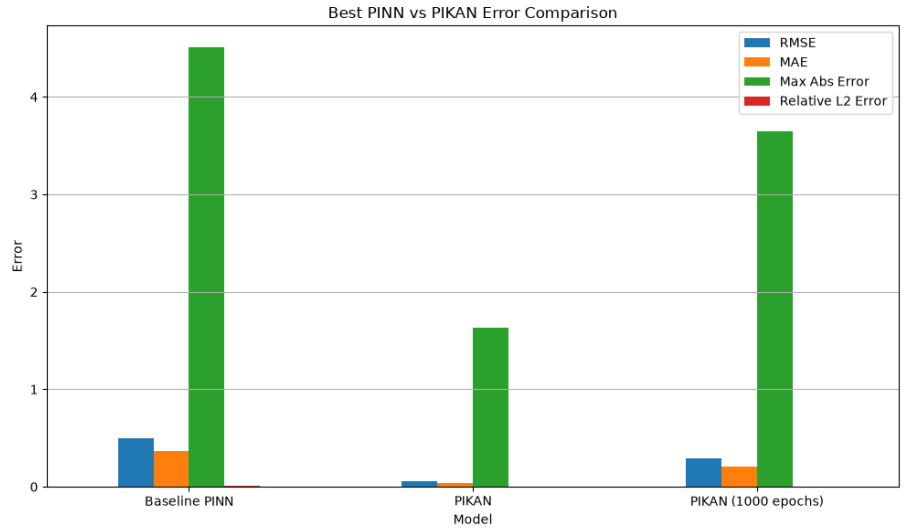


Figure 11: PINN vs PIKAN method comparison

Compared with the baseline PINN, the PIKAN achieved lower errors across all error metrics, suggesting that the KAN architecture provided a more effective function approximation for this experiment. However, this improvement came with additional computational cost.

Method	Architecture / Grid	Params	Epochs	Training Points	Eval. Grid	Runtime	Rt./Epoch	MSE	RMSE	MAE	Max Error	Rel. L^2
Crank-Nicolson	$M = 100$, $N = 100$	0	N/A	N/A	100×50	0.29	N/A	0.000096	0.009815	0.003887	0.129387	0.000102
Baseline PINN	$[2, 32, 32, 32, 1]$	2241	5000	$N_f = 500$, $N_{ic} = 200$, $N_{bc} = 200$	100×50	33.70	0.006470	0.250624	0.500623	0.363268	4.505993	0.005180
PIKAN	$[2, 16, 16, 1]$, grid $= 5$, degree $= 3$	4256	3000	$N_f = 300$, $N_{ic} = 150$, $N_{bc} = 150$	100×50	257.60	0.085867	0.003373	0.058075	0.033826	1.629618	0.000601

Table 7: Final comparison of Crank-Nicolson, baseline PINN and PIKAN methods.

Overall, Crank–Nicolson achieved the best accuracy-runtime trade-off. It produced highly accurate prices with very low computational cost. The PINN and PIKAN were both able to approximate the solution, but required substantially longer runtimes due to training. The PIKAN improved on the baseline PINN in terms of accuracy, although it did not outperform Crank-Nicolson in this one-dimensional Black-Scholes setting.

5.5 Volatility Sensitivity Analysis

Method	σ	Runtime	MSE	RMSE	MAE	Max Error	Rel. L^2
Crank-Nicolson	0.1	0.0133	0.000013	0.003605	0.001399	0.014840	0.000081
Baseline PINN	0.1	43.6705	0.434965	0.659518	0.599307	1.118845	0.014895
PIKAN	0.1	458.0159	0.000320	0.017876	0.012650	0.089956	0.000404
Crank-Nicolson	0.2	0.0086	0.000004	0.001936	0.000985	0.006102	0.000044
Baseline PINN	0.2	41.3267	0.277240	0.526535	0.506788	0.826542	0.011845
PIKAN	0.2	594.5218	0.000530	0.023014	0.015272	0.125084	0.000518
Crank-Nicolson	0.3	0.0162	0.000406	0.020154	0.008929	0.088164	0.000448
Baseline PINN	0.3	80.8969	0.134470	0.366701	0.358464	0.486760	0.008145
PIKAN	0.3	487.2392	0.002573	0.050726	0.030167	0.250927	0.001127
Crank-Nicolson	0.4	0.0189	0.033453	0.182902	0.088071	0.669549	0.003962
Baseline PINN	0.4	78.8728	0.297595	0.545522	0.467801	1.272574	0.011818
PIKAN	0.4	714.6205	0.052680	0.229521	0.118831	0.909407	0.004972

Table 8: Sensitivity analysis across different volatility values.

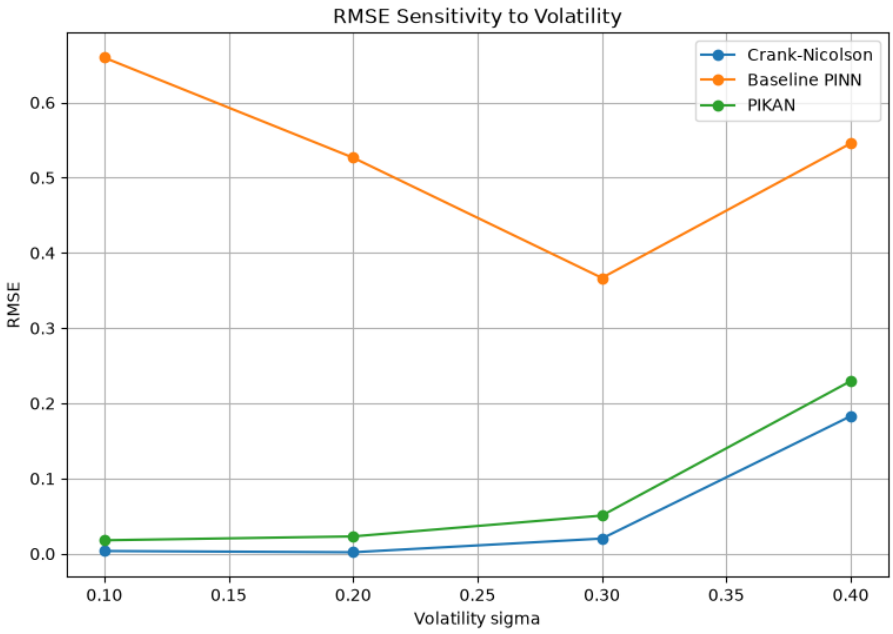


Figure 12: RMSE Sensitivity to Volatility

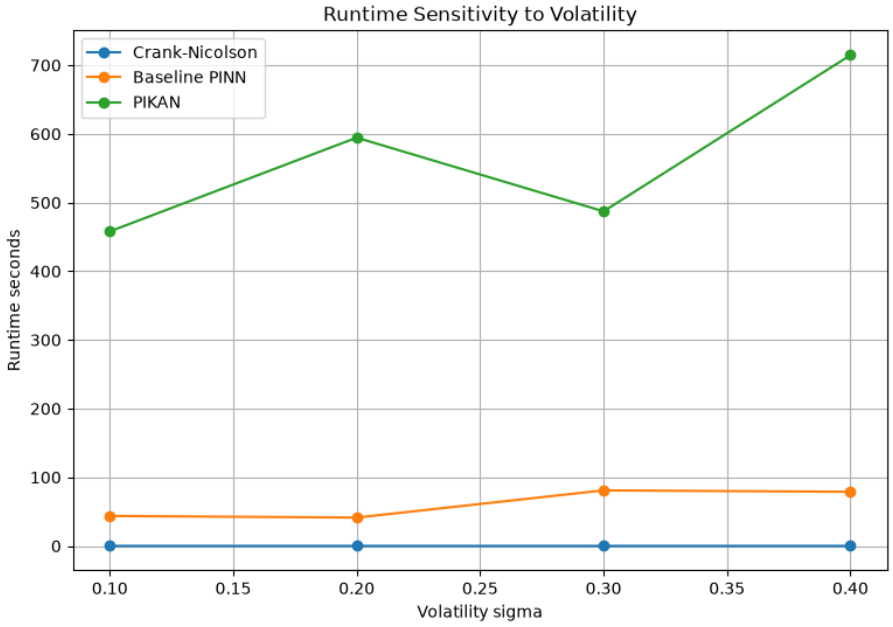


Figure 13: Runtime Sensitivity to Volatility

Table 8 and Figures 12, 13 show how the three methods behave as volatility is varied. Crank-Nicolson remains the most accurate and computationally efficient method for most volatility values. Its RMSE is very small for $\sigma = 0.1$, $\sigma = 0.2$, and $\sigma = 0.3$, although the error increases noticeably at $\sigma = 0.4$. This suggests that the fixed finite difference grid becomes less accurate as volatility increases.

The baseline PINN produces the largest RMSE across all volatility values. Its error decreases between $\sigma = 0.1$ and $\sigma = 0.3$, but rises again at $\sigma = 0.4$, showing some sensitivity to the training setup. The PIKAN performs substantially better than the baseline PINN in terms of

RMSE for every volatility tested. However, Figure 13 shows that this improvement comes at a significant computational cost, with PIKAN runtimes much larger than both the baseline PINN and Crank-Nicolson. The Crank-Nicolson runtime is so short compared to the other models that it appears as zero in Figure 13.

6 Discussion

From Section 5, it is clear that the best performing method for the Black-Scholes equation is the Crank-Nicolson method. It recorded the lowest error across all metrics while having a considerably shorter runtime than the neural network architectures. Given the Black-Scholes equation is a low-dimensional parabolic PDE this is unsurprising, as the Crank-Nicolson method is commonly employed to solve such equations.

While the PINN did capture the profile of the Black-Scholes price surface and recorded reasonable errors, these errors were considerably higher than the errors from the Crank-Nicolson method. With an equation as simple as the Black-Scholes equation, which is one-dimensional and has an analytical solution, using PINNs to solve it was inefficient in these experiments. However the PINN framework has the potential to become more competitive when considering higher-dimensional problems, for example the Heston model, where classical grid-based methods become more expensive.

The PIKAN improved accuracy over the PINN considerably, though it could not replicate the errors of the Crank-Nicolson method. In exchange, it required additional training time over the PINN and used nearly twice as many parameters. However when trained for only 1000 epochs, it still recorded lower errors than the PINN with a shorter runtime than when trained for 3000 epochs.

Overall, as hinted at before, these neural network architectures may be more competitive with Crank-Nicolson when considering higher-dimensional problems, or problems where implementing a classical grid becomes expensive.

7 Limitations

This project only considered European calls under the Black-Scholes equation, and there is far more to option pricing than is covered in the scope of this project. Black-Scholes was used as a controlled benchmark; its use in this project is not intended as a real-world pricing tool.

This project considered a simplistic model with no stochastic volatility, dividends or the potential for early exercise.

The comparison between the neural methods should also be interpreted carefully, since the PINN and PIKAN models did not have identical architectures, parameter counts, training points or epoch counts. The results therefore compare the implementations used in this project rather than proving that one architecture will always outperform the other.

The runtime results are hardware-dependent. All experiments were run on a laptop with a 12th Gen Intel Core i5-1235U processor, 16 GB RAM, and Intel Iris Xe integrated graphics, without a dedicated CUDA-enabled GPU. Consequently, the reported PINN and PIKAN training

times may be higher than those obtained on GPU-accelerated hardware.

8 Conclusion

The results show that Crank-Nicolson remains the most efficient and reliable method for the one-dimensional Black-Scholes problem considered in this project. The PINN and PIKAN models were able to approximate the option price surface, with PIKAN improving on the baseline PINN in all error metrics. However, these neural methods required substantially greater training time and were more sensitive to implementation choices. Overall, the project suggests that physics-informed neural networks are promising flexible PDE solvers, but for this classical low-dimensional pricing problem, traditional finite difference methods remain superior in accuracy-efficiency trade-off.

References

M. Raissi, P. Perdikaris and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations", *Journal of Computational Physics*, 378, pp. 686–707, 2019.

J. D. Toscano, V. Oommen, A. J. Varghese, Z. Zou, N. Ahmadi Daryakenari, C. Wu and G. E. Karniadakis, "From PINNs to PIKANs: Recent advances in physics-informed machine learning", *Machine Learning for Computational Science and Engineering*, 2025.

K. Shukla, J. D. Toscano, Z. Wang, Z. Zou and G. E. Karniadakis, "A comprehensive and FAIR comparison between MLP and KAN representations for differential equations and operator networks", *Computer Methods in Applied Mechanics and Engineering*, 431, 117290, 2024.

A. Dhiman and Y. Hu, "Physics informed neural network for option pricing", *arXiv preprint arXiv:2312.06711*, 2023.

I. E. W. Putri and R. Artiono, "Numerical pricing of European stock option based on Black-Scholes model using Crank-Nicolson method", *ZERO: Jurnal Sains, Matematika dan Terapan*, 2026.